

## Operating System

### Lab Assignment- 6

#### Question 1 — Banker's Algorithm Simulation

Implement the **Banker's Algorithm** for deadlock avoidance as a user-space xv6 program.

##### Requirements:

1. Write `bankers.c` that models `n` processes and `m` resource types using the standard data structures:
  - `Allocation[n][m]` — resources currently allocated to each process.
  - `Max[n][m]` — maximum demand of each process.
  - `Available[m]` — currently available instances of each resource type.
  - `Need[m] = Max - Allocation` (computed at runtime).
2. Hardcode a sample scenario (at least 5 processes, 3 resource types) inside the program, similar to the classic textbook example.
3. Implement the **Safety Algorithm**: determine whether the system is in a safe state, and if so, print a valid **safe sequence** of process execution.
4. Implement the **Resource Request Algorithm**: accept a resource request `Request[i]` from a process `P_i` (can be hardcoded or read via command-line args using xv6's `argv`), and:
  - Check if the request exceeds `Need[i]` or `Available` — reject if so.
  - Simulate granting the request, re-run the safety algorithm, and either commit the allocation (if still safe) or roll it back and print "**Request denied — would lead to unsafe state**".
5. Test with **at least two scenarios**: one where the request is granted safely, and one where it must be denied.

**Deliverable:** `bankers.c` + output log showing the safe sequence and both the granted and denied request scenarios.

#### Question 2 — Deadlock Detection using Resource Allocation Graph

Implement a **deadlock detection algorithm** based on the Resource Allocation Graph (RAG) / wait-for graph model.

##### Requirements:

1. Write `deadlockdetect.c` that represents:
  - A set of processes and resource instances.

- An **allocation matrix** (which process holds which resource) and a **request matrix** (which process is waiting for which resource).
- 2. Construct a **wait-for graph** from the allocation and request matrices (an edge  $P_i \rightarrow P_j$  exists if  $P_i$  is waiting on a resource held by  $P_j$ ).
- 3. Implement a **cycle-detection algorithm** (e.g., DFS-based) over the wait-for graph to determine whether a deadlock exists.
- 4. If a deadlock is detected, print the exact **cycle of processes** involved (e.g.,  $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ ).
- 5. Test your program with **two hardcoded scenarios**:
  - One with **no deadlock** (acyclic wait-for graph).
  - One **with a deadlock** (at least one cycle), including a case with a **circular wait among 3 or more processes** to show it isn't limited to 2-process cycles.

**Deliverable:** `deadlockdetect.c` + output log clearly showing graph construction, detection result, and the printed cycle when one exists.

### Question 3 — Deadlock Prevention via Resource Ordering

Demonstrate **deadlock prevention** using the **resource-ordering (hierarchical allocation)** technique, actually running as real xv6 processes (not just a simulation).

#### Requirements:

1. Create at least **2 shared resources** in xv6 (e.g., two locks — you may use xv6's `sleeplock/spinlock` directly, or a custom semaphore syscall from a previous assignment).
2. Write `resourceorder.c` that:
  - First demonstrates the **problem**: fork 2 processes where Process A acquires `Lock1` then tries `Lock2`, while Process B acquires `Lock2` then tries `Lock1` — introduce a deliberate delay between the two acquisitions (e.g., a busy-wait loop) to make the deadlock reliably reproducible. Show (with a timeout or a manual interrupt note in your log) that this configuration **hangs**.
  - Then demonstrates the **fix**: modify both processes so that **every process acquires locks in the same global order** (e.g., always `Lock1` before `Lock2`), and show that the same workload now **completes without hanging**.
3. Clearly print timestamps or step markers (e.g., "`Process A acquired Lock1`", "`Process A waiting for Lock2`") so the deadlock (and its absence after the fix) is visible in the output log.

**Deliverable:** `resourceorder.c` (or two versions: `resourceorder_bad.c` and `resourceorder_fixed.c`) + output logs for both the deadlocking and fixed runs + a short written explanation of why consistent ordering eliminates circular wait.

### Question 4 — Combined Synchronization & Deadlock Avoidance

Extend the **Dining Philosophers** setup to a more general **multi-resource synchronization problem** that requires both correct synchronization *and* an explicit deadlock-avoidance strategy.

#### Requirements:

1. Model **5 processes**, each of which needs **2 out of 3 shared resource types** (e.g., a "printer," a "scanner," and a "disk") to complete a task, with each resource having limited instances (e.g., 2 printers, 1 scanner, 2 disks).
2. Implement `syncdeadlock.c` using `fork()` to create the 5 processes, using locks/semaphores to protect access to each resource pool.
3. Apply **one deadlock-avoidance or prevention strategy** of your choice (you may reuse resource ordering from Q3, or implement a Banker's-Algorithm-style admission check from Q1 before granting a resource) — clearly state and justify which strategy you used.
4. Each process should:
  - Print when it requests a resource, when it is granted one, when it starts its "work" (simulated with a short busy-wait or sleep), and when it releases its resources.
5. Run the program for enough cycles/iterations to demonstrate:
  - No process is permanently blocked (no deadlock occurs).
  - Resources are correctly shared without exceeding available instances at any point.

**Deliverable:** `syncdeadlock.c` + a short write-up identifying the strategy used and why it prevents deadlock + output log showing all 5 processes completing their work across multiple cycles without hanging.